

Logické programovanie a inferencia (LPI)

Odporúčací systém filmov, seriálov, kníh a videohier v jazyku

Prolog

s webovým playgroundom a HTTP API

Téma 9: Program odporúčajúci filmy/knihy/technológie

Tím — Jozef Belušák, Ján Kapurík, Adam Peško, Robert Kardoš, Branislav Zurian, Dávid Tkáč
Technická univerzita v Košiciach, Fakulta elektrotechniky a informatiky

máj 2026

Abstrakt

V práci predstavujeme *vysvetliteľný odporúčací systém* implementovaný v jazyku SWI-Prolog. Systém odporúča štyri prepojené typy obsahu — **filmy, seriály, knihy a videohry** — nad spoločným dátovým modelom `polozka/4`. Odporúčania vznikajú kombináciou content-based pravidiel, skórovania, collaborative filtering, cross-domain odporúčania, grafu vzťahov medzi položkami a constraint výberu filmového večera do časového limitu. Hlavnou hodnotou riešenia je transparentnosť: pre každé odporúčanie systém vie vygenerovať konkrétne dôvody zložené z faktov a pravidiel.

Aktualizovaná verzia projektu obsahuje aj webový frontend v Next.js a tenkú HTTP JSON API vrstvu nad Prolog jadrom. Frontend slúži ako interaktívny playground: používateľ si vyberie profil, zobrazí odporúčania, hybridné skóre, podobných používateľov, cross-domain väzby, fanúšikov položky, graf vzťahov, filmový večer a štatistiky. Nasadená verzia aplikácie je dostupná na adrese <https://prolog-recommendation-system.up.railway.app/>.

1 Motivácia problému

Odporúčacie systémy patria medzi najbežnejšie nasadené aplikácie umelej inteligencie. Stretávame sa s nimi denne — Netflix nám navrhuje filmy, Spotify skladby, Amazon produkty, YouTube ďalšie video. Komerčné systémy dnes prevažne stavajú na štatistických modeloch a neurónových sieťach. Tieto prístupy dosahujú výbornú presnosť, no majú jednu zásadnú slabinu — sú **netransparentné**. Používateľ dostane odporúčanie, ale nikto presne nevie, prečo práve toto. Vznikajú otázky o spravodlivosti, manipulácii a *filter bubble* fenoméne.

V poslednom čase narastá záujem o *vysvetliteľnú umelú inteligenciu* (*explainable AI*, XAI). A práve tu má **logické programovanie** prirodzenú výhodu: každé odporúčanie možno spätne odôvodniť reťazcom použitých pravidiel. Prolog predstavuje historicky overený nástroj, v ktorom je modelovanie pravidlami prvotriednou občianskou kategóriou — nie dodatočnou nadstavbou.

Náš projekt ide o krok ďalej než typické filmové demo. Doménu sme rozšírili o **seriály, knihy a videohry**, modelované cez jednotný predikát `polozka/4`. To otvára celkom novú triedu dopytov, ktoré klasický filmový odporúčateľ nevie:

- *Páči sa mi film Dune* $\Rightarrow$ odporučiť mi knihu Dune (filmová adaptácia rovnakého románu, spoločné fiktívne univerzum), prípadne hru Mass Effect (zdieľaný žáner sci-fi).
- *Kto všetko by mal rád hru The Witcher 3?* — reverzný filter ponad pravidlo `odporucam/2`.
- *Z filmu Dune sa cez graf zdieľaných atribútov (žáner $\rightarrow$ krajina $\rightarrow$ tvorca) dostať k hre Dark Souls.*

- *Vyber mi filmový večer do 300 minút* — optimalizácia nad odporúčanými filmami cez CLP(FD).

Aktualizovaná verzia projektu už nie je iba konzolový Prolog program. Nad jadrom vznikla HTTP JSON API vrstva a webový frontend, ktorý sprístupňuje rovnaké predikáty cez prehľadné používateľské rozhranie. Tým sa projekt mení z demonštračnej znalostnej bázy na interaktívny playground vhodný aj na živú prezentáciu.

2 Aktuálny stav v oblasti

Existujú tri hlavné triedy odporúčacích systémov.

Content-based filtering odporúča položky podobné tým, ktoré sa používateľovi páčili v minulosti. Pracuje s atribútmi položiek (žáner, autor, kľúčové slová). Výhodou je nezávislosť od ostatných používateľov a dobrá vysvetliteľnosť. Nevýhodou je obmedzená diverzita — systém má sklony zacykliť používateľa do *filter bubble*.

Collaborative filtering odporúča to, čo sa páčilo iným podobným používateľom. Nepotrebuje atribúty položiek, stačia hodnotenia. Je to prístup, na ktorom je postavený napr. pôvodný Netflix Prize algoritmus [6]. Slabou stránkou je *cold start problem* — nového používateľa alebo novú položku systém nevie kam zaradiť.

Hybridné systémy kombinujú oba prístupy a v praxi dnes dominujú. Moderné implementácie (YouTube, TikTok, Spotify) k tomu pridávajú hlboké neurónové siete, ktoré z histórie interakcií učia latentné reprezentácie.

Logické a *knowledge-based* prístupy tvoria menšinovú, no dobre etablovanú vetvu [3, 4]. Sú obzvlášť úspešné v doménach, kde používateľ kupuje položku zriedka (autá, dovolenky, hypotéky) a kde je dôležité formálne odôvodnenie. Náš projekt patrí do tejto kategórie.

3 Teoretické východiská

3.1 Logické programovanie a Hornove klauzuly

Logické programovanie vychádza z fragmentu predikátovej logiky prvého rádu — **Hornových klauzúl**. Hornova klauzula má najviac jeden pozitívny literál a v Prologu sa zapisuje v tvare

```
hlava :- telo1, telo2, ..., teloN.
```

čo zodpovedá logickej implikácii $telo1 \wedge \dots \wedge teloN \Rightarrow hlava$.

3.2 SLD-rezolúcia a unifikácia

Prolog dokazuje cieľ pomocou **SLD-rezolúcie** (*Selective Linear resolution for Definite clauses*). Pri každom kroku vyberie jeden cieľ z pracovnej množiny, nájde klauzulu, ktorej hlavu možno **unifikovať** s týmto cieľom, a nahradí cieľ telom danej klauzuly [5].

3.3 Backtracking a hľadanie všetkých riešení

Prolog používa stratégiu **prehľadávania do hĺbky s backtrackingom**. Ak nejaký podcieľ zlyhá, systém sa vráti k poslednému miestu, kde existovali ďalšie alternatívy (*choice point*), a vyskúša ich. Vďaka tomu jeden Prolog dopyt môže priniesť postupne **všetky** riešenia — vlastnosť, ktorú v iných paradigmách musíme manuálne emulovať generátormi alebo kolekciami.

3.4 Negácia ako neúspech (NAF)

Klasická logika nedovoľuje z neprítomnosti dôkazu odvodiť negáciu. Prolog používa pragmatický variant — *negation as failure* (\+): cieľ \+ G uspeje práve vtedy, keď sa nepodari dokázať G. Tento princíp je nutný napr. pre vyjadrenie „používateľ ešte nekonzumoval položku“:

```
este_nekonzumoval(P, F) :- \+ uz_konzumoval(P, F).
```

Ide o tzv. *closed world assumption* — predpokladáme, že znalostná báza obsahuje všetko relevantné a všetko ostatné je nepravdivé.

4 Popis riešenia

4.1 Architektúra

Systém má štyri logické vrstvy:

1. **Znalostná báza** — fakty o 55 položkách (15 filmov, 10 seriálov, 15 kníh, 15 hier) a 5 používateľoch (`backend/program.pl`, sekcia 1).
2. **Inferenčné pravidlá** — pomocné predikáty, hlavné odporúčanie, skórovanie, vysvetľovanie, collaborative filtering, cross-domain odporúčanie, filter *fanúšikovia*, graf príbuznosti, adaptácie, univerzá, hybridné skóre a constraint odporúčanie (`backend/program.pl`, sekcie 2A–2G).
3. **HTTP API** — tenká JSON vrstva `backend/api.pl` nad `program.pl`; každý handler pozbera riešenia cez `findall/setof` a pošle `reply_json_dict`. API beží cez `library(http/thread_httpd)` a štandardne počúva na porte 4000 alebo na porte z premennej prostredia `PORT`.
4. **Webová aplikácia** — interaktívny playground v Next.js 15 (`web/`); všetka odporúčacia logika zostáva v Prologu, frontend robí výber používateľa, volania endpointov, vizualizáciu výsledkov a demo rozhranie. Nasadená verzia je dostupná na <https://prolog-recommendation-system.up.railway.app/>.

Vrstva	Technológia	Zodpovednosť
Frontend	Next.js 15, React 19, Tailwind CSS	UI, karty, graf, interakcie
API	SWI-Prolog HTTP knižnice	JSON endpointy nad predikátmi
Jadro	SWI-Prolog + CLP(FD)	znalostná báza a inferencia
Nasadenie	Docker, Railway	spustenie backendu a frontendu

4.2 Unifikovaný dátový model položka/4

Kľúčovým návrhovým rozhodnutím je **zjednotenie štyroch domén** (film, seriál, kniha, hra) pod jeden predikát:

```
1 % polozka(+Id, +Typ, +Nazov, +Rok)
2 %   Typ in {film, serial, kniha, hra}
3 polozka(inception, film, 'Inception', 2010).
4 polozka(breaking_bad, serial, 'Breaking Bad', 2008).
5 polozka(dune_book, kniha, 'Dune', 1965).
6 polozka(witcher3, hra, 'The Witcher 3: Wild Hunt', 2015).
```

Listing 1: Reprezentácia položiek

Spoločné atribúty platia pre všetky typy:

Predikát	Aritnosť	Význam
zaner/2	(Id, Žáner)	žáner položky
hodnotenie/2	(Id, Skóre)	hodnotenie 1–10
krajina/2	(Id, Krajina)	krajina pôvodu
studio/2	(Id, Štúdio)	štúdio / vydavateľstvo / platforma
tvorca/3	(Id, Rola, Meno)	režisér / showrunner / autor / vývojár

Typovo-špecifické sú len rozmery: `dlzka/2` (minúty pre filmy), `epizody/2` (pre seriály), `strany/2` (pre knihy) a `dlzka_hry/2` (hodiny pre hry). Tieto sa konzumujú jediným polymorfným filtrom `vyhovuje_rozmeru/3`, ktorý dispatchuje podľa typu.

4.3 Hlavné inferenčné pravidlo

```

1 odporucam(P, Polozka) :-
2   polozka(Polozka, Typ, _, _),
3   pouzivatel_chce_typ(P, Typ),
4   once(vyhovuje_zaneru(P, Polozka)),
5   vyhovuje_hodnoteniu(P, Polozka),
6   vyhovuje_rozmeru(P, Polozka, Typ),
7   este_nekonzumoval(P, Polozka).
```

Pravidlo hovorí, že položka je odporúčaná používateľovi vtedy, keď simultánne platí: typ patrí medzi obľúbené, žáner sa zhoduje, hodnotenie je dostatočne vysoké, rozmer (dĺžka filmu / počet strán / dĺžka hry) je v limite, a používateľ položku ešte nekonsumoval. Operátor `once/1` zabezpečí, že každú položku vráti backtracking iba raz, aj keď je žánrov spoločných viacero.

4.4 Skórovanie a zoradovanie

```

1 skore_odporucania(P, Polozka, Skore) :-
2   hodnotenie(Polozka, H),
3   ( vyhovuje_zaneru(P, Polozka) -> BZ = 2 ; BZ = 0 ),
4   ( bonus_tvorcu(P, Polozka) -> BT = 3 ; BT = 0 ),
5   ( bonus_krajiny(P, Polozka) -> BK = 1 ; BK = 0 ),
6   ( bonus_studia(P, Polozka) -> BS = 1 ; BS = 0 ),
7   Skore is H + BZ + BT + BK + BS.
```

Skóre je súčet hodnotenia (1–10), bonusu za žáner (+2), za obľúbeného tvorca (+3), za krajinu (+1) a za štúdio (+1). Maximum je teda 17.

4.5 Vysvetľovanie odporúčaní

Hlavnou pridanou hodnotou logického prístupu je **transparentnosť**. Predikát `dovody_odporucania/3` najprv vráti dôvody ako čisté Prolog termy (typ, žáner, hodnotenie, rozmer, tvorca, krajina a štúdio). Predikát `vysvetli/2` je potom už len prezentačná vrstva, ktorá tieto dôvody vypíše používateľovi. Tým sú vysvetlenia zároveň testovateľné v `PLUnit` testoch.

4.6 Collaborative filtering

Pôvodné jednoduché pravidlo „zdieľajú aspoň dva žánre“ sme rozšírili na skórovanú podobnosť používateľov. Predikát `podobnost_pouzivatelov/4` porovná spoločné žánre, typy obsahu, tvorcov, krajiny a štúdiá; žáner a tvorca majú vyššiu váhu ako pomocné atribúty. Používatelia sú považovaní za podobných, keď dosiahnu skóre aspoň 4.

```

1 podobni_pouzivatelia(P1, P2) :-
2     podobnost_pouzivatelov(P1, P2, Skore),
3     Skore >= 4.

```

Samotné odporucam_collaborative/2 už navyše rešpektuje obľúbený typ cieľovej položky aj typovo špecifický rozmerový limit.

4.7 Cross-domain odporúčanie

Hlavná pridaná hodnota oproti pôvodnému filmovému demu je *prepojenie domén*:

```

1 cross_domain_odporucam(P, Zdroj, Ciel, Dovod) :-
2     uz_konzumoval(P, Zdroj),
3     polozka(Zdroj, TypZdroj, _, _),
4     polozka(Ciel, TypCiel, _, _),
5     TypZdroj \= TypCiel,
6     pouzivatel_chce_typ(P, TypCiel),
7     este_nekonzumoval(P, Ciel),
8     vyhovuje_hodnoteniu(P, Ciel),
9     vyhovuje_rozmeru(P, Ciel, TypCiel),
10    cross_dovod(Zdroj, Ciel, Dovod).
11
12 cross_dovod(Z, C, spolocny_zaner(X))    :- zaner(Z,X), zaner(C,X).
13 cross_dovod(Z, C, spolocny_tvorca(M))   :- tvorca(Z,_,M), tvorca(C,_,M).
14 cross_dovod(Z, C, spolocna_krajina(K))  :- krajina(Z,K), krajina(C,K).
15 cross_dovod(Z, C, spolocne_studio(S))   :- studio(Z,S), studio(C,S).

```

Predikát `cross_dovod/3` formálne zachytáva most medzi dvomi položkami z rôznych domén — spoločným žánrom, tvorcom, krajinou alebo štúdiom. Backtracking automaticky vygeneruje všetky aplikovateľné mosty.

4.8 Filter „kto má rád X“ (fanúšikovia)

```

1 fanusikovia(Polozka, Pouzivatelia) :-
2     findall(P,
3         ( pouzivatel(P, _),
4           ( odporucam(P, Polozka)
5             ; uz_konzumoval(P, Polozka) ) ),
6         Zoznam),
7     sort(Zoznam, Pouzivatelia).

```

Toto je *reverzný dopyt* nad pravidlom `odporucam/2`. Tam, kde štandardný odporúčač pre zadaného používateľa hľadá položky, my pre zadanú položku hľadáme používateľov. Rozšírená verzia `fanusikovia_s_dovodmi/2` ku každému fanúšikovi priloží aj konkrétne dôvody zhody.

4.9 Cestovanie po grafe vzťahov

Položky v znalostnej báze tvoria graf prepojený cez zdieľané atribúty. Predikát `pribuzne_polozky/3` určuje silu prepojenia ako počet rôznych zdieľaných atribútov:

```

1 pribuzne_polozky(P1, P2, Sila) :-
2     polozka(P1,_,_,_), polozka(P2,_,_,_), P1 \= P2,
3     findall(A, zdielany_atribut(P1, P2, A), AtrZoznam),
4     list_to_set(AtrZoznam, Spolocne),
5     length(Spolocne, Sila),
6     Sila > 0.

```

Tranzitívna verzia `retazec/4` vie nájsť cestu medzi dvomi položkami v grafe príbuznosti do zadaného počtu krokov. Vďaka backtrackingu a detekcii cyklov (`\+ member(..., Acc)`) je hľadanie kompletne a bezpečné.

4.10 Adaptácie, univerzá a hybridné skóre

Doteraz sme prepájali domény len cez zdieľané *atribúty* (žáner, krajina, štúdio, tvorca). Tento prístup však necháva nezachytené dve silné prepojenia: explicitnú **adaptáciu** jedného diela na druhé (film podľa knihy, hra podľa knihy) a **spoločné fiktívne univerzum** (Tolkienovo Stre dozemie obsahuje aj *Hobbita*, aj *Pán prsteňov*). Tieto prepojenia patria do znalostnej bázy ako samostatné fakty:

```
1 adaptacia(dune,      dune_book).      % film 2021 podľa knihy 1965
2 adaptacia(witcher3, witcher_book).    % hra podľa Sapkowskeho knih
3
4 univerzum(dune,      dune_universe).
5 univerzum(dune_book, dune_universe).
6 univerzum(witcher3,  witcher_universe).
7 univerzum(witcher_book, witcher_universe).
8 univerzum(hobbit,    middle_earth).
9 univerzum(lotr,      middle_earth).
```

Listing 2: Adaptácie a univerzá

Cross-domain odporúčač sa rozšíri o dva nové *mosty* — stačí pridať klauzuly k existujúcemu predikátu `cross_dovod/3`:

```
1 cross_dovod(Z, C, spolocne_univerzum(U)) :-
2     univerzum(Z, U), univerzum(C, U).
3
4 cross_dovod(Z, C, adaptacia) :-
5     ( adaptacia(Z, C) ; adaptacia(C, Z) ).
```

Hybridné skóre potom kombinuje content-based skóre s ďalšími tromi signálmi:

```
1 hybridne_skore(P, Polozka, Skore) :-
2     skore_odporucania(P, Polozka, Content),
3     ( bonus_collab(P, Polozka)    -> BC = 3 ; BC = 0 ),
4     ( bonus_adaptacia(P, Polozka) -> BA = 2 ; BA = 0 ),
5     ( bonus_univerzum(P, Polozka) -> BU = 2 ; BU = 0 ),
6     Skore is Content + BC + BA + BU.
```

Bonusy modelujú tri intuície: (1) *collaborative* — „niektorý podobný používateľ to už konzumoval“; (2) *adaptácia* — „už si videl iný formát toho istého diela“; (3) *univerzum* — „už si konzumoval iné dielo z tohto sveta“. Maximum hybridného skóre je teda $10+2+3+1+1+3+2+2 = 24$.

4.11 Constraint odporúčanie cez CLP(FD)

Okrem jednotlivých odporúčaní systém obsahuje aj malý optimalizačný problém: `filmovy_vecer/5`. Zo všetkých filmov odporúčaných danému používateľovi vyberie neprázdny podzoznam tak, aby sa zmestil do zadaného časového limitu a maximalizoval súčet skóre.

```
1 filmovy_vecer(P, MaxMinut, Vyber, Dlzka, Skore) :-
2     ...,
3     scalar_product(Dlzkys, Bitys, #=, Dlzka),
4     scalar_product(SkoreFilm, Bitys, #=, Skore),
5     Dlzka #=< MaxMinut,
6     sum(Bitys, #>=, 1),
7     labeling([max(Skore)], Bitys).
```

Tento príklad ukazuje, že Prolog nemusí len filtrovať fakty, ale vie riešiť aj kombinatorické úlohy s tvrdými obmedzeniami.

4.12 Webová aplikácia a HTTP API

Ako doplnok k príkazovému riadku sme implementovali interaktívny webový *playground*. Vďaka nemu nie je potrebné ručne písať Prolog dopyty v termináli: používateľ si vyberie profil a jednotlivé sekcie aplikácie volajú Prolog backend cez JSON endpointy.

Backend (`backend/api.pl`) je tenká HTTP JSON vrstva nad nezmeneným `program.pl`. Používa štandardné knižnice SWI-Prologu `library(http/thread_httpd)`, `library(http/http_dispatch)`, `library(http/http_json)` a `library(http/http_cors)`. API vystavuje 13 endpointov:

Endpoint	Úloha
<code>/api/health</code>	kontrola dostupnosti backendu
<code>/api/users</code>	zoznam používateľov a ich profilov
<code>/api/items</code>	zoznam položiek zo znalostnej bázy
<code>/api/recommend</code>	content-based odporúčania
<code>/api/explain</code>	dôvody odporúčania konkrétnej položky
<code>/api/hybrid</code>	hybridné skóre
<code>/api/collaborative</code>	podobní používatelia a ich odporúčania
<code>/api/cross-domain</code>	odporúčanie medzi rôznymi typmi obsahu
<code>/api/fans</code>	reverzný filter fanúšikov položky
<code>/api/related</code>	príbuzné položky v grafe
<code>/api/path</code>	cesta medzi položkami v grafe vzťahov
<code>/api/movie-night</code>	výber filmového večera do limitu
<code>/api/stats</code>	štatistiky znalostnej bázy

Každý handler žiadnu odporúčaciu logiku neduplikuje. Iba pripraví parametre, zavolá príslušný predikát z `program.pl`, pozbiera riešenia cez `findall` alebo `setof` a prevedie výsledok na JSON pomocou `reply_json_dict`. Štruktúrované dôvody odporúčania sa prevádzajú na JSON objekty, aby ich frontend vedel zobrazit ako čitateľné vysvetlenia.

Frontend (`web/`) je aplikácia v Next.js 15 (App Router), React 19 a Tailwind CSS. Stránka je renderovaná dynamicky (`force-dynamic`), aby sa údaje vždy čítali zo živého Prolog backendu. Súčasťou rozhrania sú komponenty:

- `UserSwitcher` — výber používateľského profilu,
- `RecommendSection` — content-based odporúčania so skóre,
- `HybridSection` — hybridné skóre,
- `CollaborativeSection` — podobní používatelia,
- `CrossDomainSection` — odporúčanie medzi doménami,
- `FansSection` — fanúšikovia konkrétnej položky,
- `RelationGraph` — vizualizácia vzťahov medzi položkami,
- `MovieNightSection` — constraint výber filmového večera,
- `StatsSection` — štatistiky znalostnej bázy.

Aplikácia je dostupná lokálne cez Docker Compose:

```
docker compose up --build
```

Frontend beží lokálne na `http://localhost:3000`, backend na `http://localhost:4000`. Verejne nasadená verzia playgroundu je dostupná na adrese:

<https://prolog-recommendation-system.up.railway.app/>

Webová vrstva tak robí z Prolog programu použiteľné demo: ukazuje profily používateľov, odporúčania, vysvetlenia, hybridné skóre, cross-domain väzby, graf aj optimalizačný výber filmového večera.

5 Ukážky použitia

Všetky výstupy boli získané spustením `swipl -q -g run_all -t halt examples.pl`.

5.1 Príklad 1 — základné odporúčanie

```
?- odporucam(jan, P).
-> [film]   Interstellar (2014)
-> [film]   Parasite (2019)
-> [film]   Joker (2019)
-> [film]   Dune (2021)
-> [film]   The Shining (1980)
-> [film]   Arrival (2016)
-> [film]   Blade Runner (1982)
-> [kniha]   Dune (1965)
-> [kniha]   Foundation (1951)
-> [kniha]   Hyperion (1989)
-> [kniha]   Neuromancer (1984)
-> [kniha]   The Martian (2011)
-> [kniha]   Gone Girl (2012)
-> [serial]  Breaking Bad (2008)
-> [serial]  Stranger Things (2016)
-> [serial]  The Expanse (2015)
-> [serial]  Chernobyl (2019)
-> [serial]  True Detective (2014)
-> [serial]  Westworld (2016)
-> [serial]  The Mandalorian (2019)
```

5.2 Príklad 3 — vysvetlenie

```
?- vysvetli(jan, interstellar).
Polozka: Interstellar (film, 2014)
- Typ: film (pouzivatel ho chce)
- Zaner: sci-fi (oblubeny)
- Hodnotenie: 9/10 (minimum 8)
- Dlzka filmu: 169 min (maximum 170)
- reziser: nolan (oblubeny)
- Krajina: usa (oblubena)
```

Toto je presne ten typ výstupu, aký moderné komerčné systémy len ťažko poskytujú — každé odporúčanie je odôvodnené konkrétnymi pravidlami.

5.3 Príklad 6 — cross-domain (z filmu na hru alebo seriál)

```
?- cross_domain_odporucam(peter, dune, C, D).
-> [serial] Arcane (spolocna_krajina(usa))
-> [serial] Arcane (spolocny_zaner(akcia))
-> [hra] BioShock (spolocna_krajina(usa))
-> [hra] BioShock (spolocny_zaner(akcia))
```



```

-> [hra] BioShock (spolocny_zaner(sci-fi))
-> [serial] Black Mirror (spolocny_zaner(sci-fi))
-> [serial] Breaking Bad (spolocna_krajina(usa))
-> [serial] Chernobyl (spolocna_krajina(usa))
-> [hra] Cyberpunk 2077 (spolocny_zaner(sci-fi))
-> [serial] Game of Thrones (spolocna_krajina(usa))
-> [hra] Half-Life 2 (spolocna_krajina(usa))
-> [hra] Half-Life 2 (spolocny_zaner(akcia))
-> [hra] Half-Life 2 (spolocny_zaner(sci-fi))
-> [hra] Horizon Zero Dawn (spolocny_zaner(akcia))
-> [hra] Horizon Zero Dawn (spolocny_zaner(sci-fi))
-> [hra] The Last of Us (spolocna_krajina(usa))
-> [hra] The Last of Us (spolocny_zaner(akcia))
-> [serial] The Mandalorian (spolocna_krajina(usa))
-> [serial] The Mandalorian (spolocny_zaner(akcia))
-> [serial] The Mandalorian (spolocny_zaner(sci-fi))
-> [hra] Mass Effect (spolocny_zaner(sci-fi))
-> [hra] Portal 2 (spolocna_krajina(usa))
-> [hra] Portal 2 (spolocny_zaner(sci-fi))
-> [hra] Red Dead Redemption 2 (spolocna_krajina(usa))
-> [hra] Red Dead Redemption 2 (spolocny_zaner(akcia))
-> [serial] The Expanse (spolocna_krajina(usa))
-> [serial] The Expanse (spolocny_zaner(sci-fi))
-> [serial] True Detective (spolocna_krajina(usa))
-> [serial] Westworld (spolocna_krajina(usa))
-> [serial] Westworld (spolocny_zaner(sci-fi))

```

5.4 Príklad 7 — fanúšikovia hry

```

?- fanusikovia(witcher3, U).
U = [maria, tomas].

?- fanusikovia_s_dovodmi(witcher3, P).
P = [ maria-[zaner(fantasy), zaner(rpg),
          tvorca(sapkowski), krajina(polisko),
          studio(cd_projekt_red)],
      tomas-[zaner(akcia), zaner(rpg),
              krajina(polisko), studio(cd_projekt_red)] ].

```

5.5 Príklad 8 — cesta v grafe vzťahov

```

?- retazec(amelie, dark_souls, 4, Cesta).
[film] Amelie -->
[film] Forrest Gump -->
[film] Inception -->
[kniha] The Name of the Wind -->
[hra] Dark Souls

```

5.6 Príklad 9 — adaptácie, univerzá a hybridné skóre

Maria už čítala *Hobbita* a hrala *Witcher 3*. Vďaka tomu cross-domain odporúčač cez most *adaptacia* a *spolocne_univerzum* navrhne knižného *The Last Wish*, a hybridné skóre vyzdvihne aj *Lord of the Rings* (rovnaké stredoziemné univerzum):

```
?- cross_domain_odporucam(maria, witcher3, C, D),
   ( D = adaptacia ; D = spolocne_univerzum(_) ).
-> [kniha] The Last Wish (spolocne_univerzum(witcher_universe))
-> [kniha] The Last Wish (adaptacia)

?- top_hybridne(maria, T).
1. [kniha] The Last Wish (content=14, hybrid=18)
2. [kniha] The Lord of the Rings (content=16, hybrid=18)
3. [hra] The Legend of Zelda: Breath of the Wild (content=12, hybrid=12)
4. [hra] Disco Elysium (content=12, hybrid=12)
5. [hra] Cyberpunk 2077 (content=12, hybrid=12)
```

Príklad ilustruje, ako sa explicitne modelované vzťahy (*adaptacia/2*, *univerzum/2*) prejavajú bez akéhokoľvek štatistického učenia — iba ako logické dôsledky nových faktov a jednej rozšírenej definície *cross_dovod*.

5.7 Príklad 10 — constraint výber filmového večera

```
?- filmy_vecer(jan, 300, Vyber, Dlzka, Skore).
Dlzka = 286,
Skore = 27,
Vyber = [interstellar, blade_runner].
```

Výsledok znamená, že z Janových odporúčaných filmov systém vybral kombináciu *Interstellar* a *Blade Runner*: spolu majú 286 minút a v limite 300 minút dosahujú najvyšší súčet skóre.

6 Diskusia výsledkov

Silné stránky. (1) Transparentnosť a vysvetliteľnosť — systém vie pre každé odporúčanie zdôvodniť, prečo bolo dané. (2) Modulárnosť — pridanie nového pravidla, nového atribútu, novej domény (napr. hudby) znamená dopísanie pár faktov, bez zásahu do existujúcej logiky. (3) Deklaratívnosť — kód popisuje čo má platiť, nie ako to spočítať. (4) Cross-domain prepojenia bezo zmeny architektúry vďaka unifikovanému dátovému modelu.

Obmedzenia. (1) Malá znalostná báza (55 položiek, 5 používateľov). V reálnom Netflixe ide o desiatky tisíc položiek a milióny používateľov. (2) Bez učenia z dát — pravidlá sú zafixované expertom. V hybridnom prístupe by Prolog mohol slúžiť ako symbolický modul nad ML predikciami. (3) Collaborative filtering používa ručne navrhnuté váhy podobnosti namiesto učeních latentných reprezentácií alebo kosínusovej podobnosti nad veľkou maticou hodnotení.

Možné rozšírenia.

1. Načítanie znalostnej bázy z CSV/JSON (`library(csv)`, `library(http/json)`).
2. Negatívne vysvetlenia: predikát `preco_neodporucam/3`, ktorý vráti dôvody zlyhania.
3. Probabilistické rozšírenie cez ProbLog — kvantifikácia neistoty v preferenciách.
4. Pridanie ďalších domén (hudba, podcasty, technológie) — vďaka unifikovanému modelu len ako nové fakty.

7 Rozdelenie práce v tíme

Člen	Zodpovednosť	Súbory / časti
Jozef Belušák	Hlavná logika odporúčania, štruktúra systému a prezentácia. Zodpovedal za návrh jadra odporúčania, prepojenie pravidiel a vysvetlenie hlavného inferenčného toku.	<code>backend/program.pl</code> , prezentácia
Ján Kapurík	Databáza položiek, žánre a atribúty obsahu. Pripravil znalostnú bázu položiek naprieč filmami, knihami, hrami a seriálmi.	<code>backend/program.pl</code> (§1)
Adam Peško	Používatelia, preferencie, limity a história konzumácie. Riešil profily používateľov a filtre, ktoré rozhodujú, čo sa používateľovi môže odporučiť.	<code>backend/program.pl</code> (§1, §2.A)
Robert Kardoš	Skórovanie, top odporúčania a hybridné skóre. Pripravil mechanizmus, ktorý z vhodných položiek skladá zoradený rebríček odporúčaní.	<code>backend/program.pl</code> (§2.B, §2.G)
Branislav Zurian	Podobní používatelia, cross-domain odporúčanie a graf vzťahov. Riešil prepojenia medzi používateľmi a položkami naprieč doménami.	<code>backend/program.pl</code> (§2.C–F)
Dávid Tkáč	Frontend, API napojenie, demo a testovanie. Zodpovedal za webový playground, JSON komunikáciu s Prolog backendom, demonštračné spustenie a testovacie scenáre.	<code>backend/api.pl</code> , <code>web/</code> , <code>tests.pl</code>

Práca prebiehala v iteráciách s krížovou kontrolou častí. Cieľom bolo, aby každá časť systému zostala samostatne vysvetliteľná: dáta, používateľské profily, hlavné pravidlá, skórovanie, cross-domain logika aj webové rozhranie.

Literatúra

- [1] Leon Sterling and Ehud Shapiro. *The Art of Prolog: Advanced Programming Techniques*. MIT Press, 2nd edition, 1994.
- [2] Ivan Bratko. *Prolog Programming for Artificial Intelligence*. Addison-Wesley, 4th edition, 2011.
- [3] Robin Burke. Knowledge-Based Recommender Systems. In *Encyclopedia of Library and Information Systems*, volume 69. Marcel Dekker, 2000.
- [4] Alexander Felfernig and Robin Burke. Constraint-Based Recommender Systems: Technologies and Research Issues. In *Proceedings of ICEC '08*, ACM, 2008.
- [5] John W. Lloyd. *Foundations of Logic Programming*. Springer, 2nd edition, 1987.
- [6] Charu C. Aggarwal. *Recommender Systems: The Textbook*. Springer, 2016.
- [7] Jan Wielemaker et al. *SWI-Prolog Reference Manual*. SWI-Prolog Consortium, 2024. <https://www.swi-prolog.org/pldoc/>
- [8] KKUI FEI TUKE. *Logické programovanie a inferencia (LPI) – materiály kurzu*. 2026. <https://kurzy.kpi.fei.tuke.sk/lpi/>